

ARES 코드 수정 보고서

BLE 명령 파이프라인 최적화 · 부팅 안정화 · DC 모터 대칭 출력 · HTTPS 사용 사유 · BATCH

명령 블록

2026-05-19 - 2026-05-20 합본

(주)코리아사이언스 / 동명대학교 게임학부

2026년 5월 19일 - 5월 20일

1 개요

본 보고서는 2026년 5월 19일과 5월 20일에 적용된 ARES 펌웨어 및 웹 코드의 변경 사항을 하나의 합본으로 정리한 자료이다. 두 일자의 변경은 형식적으로는 구분되지만 실질적으로 연결되어 있으며, 5월 19일에 적용한 BLE 명령 파이프라인 최적화의 재적용 과정에서 부팅 단계 크래시와 SYS_SET 핸들러 누락이 드러났고, 같은 흐름에서 DC 모터 출력 비대칭과 Web Bluetooth 호스팅 환경 의존성도 정리되었다.

전체 변경의 큰 줄기는 다음과 같다.

1. (2026-05-19) BLE 명령 파이프라인 최적화. "LED 1번 켜기 → LED 2번 켜기"와 같이 연속된 출력 명령 사이의 체감 지연(약 250ms)을 명령 부류별 응답 정책 분리로 약 40ms 수준까지 단축.
2. (2026-05-20) 같은 최적화를 재적용하면서 발견된 펌웨어 부팅 단계의 OLED 크래시와 SYS_SET 명령 핸들러 누락을 교정. 또한 DC 모터의 DIR+PWM 단일 변조 토폴로지에서 오는 전·후진 출력 비대칭을 듀얼 PWM brake-modulated 구성으로 근원에서 해소.

3. (2026-05-20) 학습용 웹앱 Web/main.html이 file://에서 Web Bluetooth가 동작하지 않는 원인 식별. 보안 컨텍스트의 문제가 아니라 ES module의 같은-출처 정책이 원인이며, 향후 학생 배포 시 HTTPS 호스팅(또는 localhost)이 사실상 강제됨을 명시.
4. (2026-05-20, 오후) 명령을 묶어 한 BLE 라운드트립으로 보내는 BATCH("한꺼번에 실행") Blockly 컨테이너 블록 도입과, 클론 BT05/HM-10 환경에서 멀티 청크 송수신을 안정화하기 위한 5단계 페이싱/race condition 수정.

2026-05-19의 작업이 기능적 개선이라면, 2026-05-20의 작업은 그 개선이 재현 가능하게 동작하도록 만든 안정화이다. 같은 코드라도 하나의 회피 가능한 크래시 때문에 정상이 비정상으로 보이고, 같은 모터라도 PWM 와이어링 한 가지가 사용자 경험을 크게 좌우한다.

2 2026-05-19 --- BLE 명령 파이프라인 최적화

ARES 로버의 블록 코딩 사용 중 "LED 1번 켜기 → LED 2번 켜기"와 같이 연속된 명령 사이에 체감 가능한 지연이 발생하던 문제를 분석하여, 명령당 누적 약 250ms에 달하던 지연을 출력 명령 기준 약 40ms 수준으로 줄였다. 변경 대상은 Web/commandexecutor.js와 Pico/main.py이며, 명령의 의미에 따라 응답 라운드트립을 선택적으로 생략하는 정책을 양쪽에 일관되게 도입하였다.

2.1 문제 진단

단일 LED 명령 한 번이 종단 간에 소요하는 시간 예산은 다음과 같이 구성되어 있었다.

#	위치	원인	시간
1	commandexecutor.js:199	sendData(cmd, true)로 모든 명령에 응답 대기	라운드트립
2	bluetooth.js:396	CHUNK_DELAY=50ms (constants.js:14)	50ms
3	main.py:86	RECEIVE_TIMEOUT_MS=50ms 무조건 대기	50ms
4	commandexecutor.js:213	모든 명령 후 setTimeout(100)	100ms
5	BLE GATT	connection interval (HM-10 기본)	30--70ms

표 1: 단일 LED 명령의 누적 지연(개선 전) --- 합계 약 250 ms.

정상 동작이라도 누적 지연이 약 250 ms / 명령에 달했다. 카운트 다운, 파티 조명처럼 짧은 간격의 LED 시퀀스가 “끊기는” 체감의 주된 원인이다.

2.2 변경 정책 --- 명령별 분리

명령을 두 부류로 나누어 응답 대기 여부와 cooldown을 다르게 적용한다.

부류	명령	응답 대기
Fire-and-forget	LED_ON, LED_OFF, MAIN_LED_*, [v0 v1 v2 v3 v4] (LED 패턴), MSG, CLEAR_DISPLAY, SERVO_FORWARD/BACKWARD/LEFT/RIGHT/STOP (연속), DC_FORWARD/BACKWARD/STOP (연속), GUN_FIRE	아니오
응답 대기 유지	BUZZER_ON, F, D (D초 blocking), SERVO_t*/DC_t* (N초 blocking), SLEEP, N (N초 blocking), DISTANCE, MAGNET, PING	예

표 2: 명령 부류와 응답 대기 정책. Pico에서 blocking으로 처리되거나 값을 반환하는 명령은 응답 대기를 유지하여 타이밍과 값 정합성을 보존한다.

2.3 Web 측 변경 --- Web/commandexecutor.js

핵심 코드

```
// 즉시 완료 명령 - Pico가 blocking 처리하지 않으므로 응답 대기 불필요.
// 시간 의존적(BUZZER_ON, SERVO_t*, DC_t*, SLEEP) 또는 값 반환(DISTANCE,
// MAGNET, PING)은 이 집합에 넣지 말 것. 응답 대기로 두어야 타이밍/값이
// 보장된다.
FIRE_AND_FORGET_HEADS: new Set([
  'LED_ON', 'LED_OFF',
  'MAIN_LED_ON', 'MAIN_LED_OFF',
  'MSG', 'CLEAR_DISPLAY',
  'SERVO_FORWARD', 'SERVO_BACKWARD', 'SERVO_LEFT', 'SERVO_RIGHT', 'SERVO_STOP',
  'DC_FORWARD', 'DC_BACKWARD', 'DC_STOP',
  'GUN_FIRE',
]),

_isFireAndForget(command) {
  if (command.startsWith '[')) return true; // LED 패턴 [v0 v1 v2 v3 v4]
  const head = command.split(',')[0];
```

```

    return this.FIRE_AND_FORGET_HEADS.has(head);
  },

  async sendCommand(command) {
    if (!state.isExecuting) {
      Logger.add('[중단] 실행이 중단되었습니다', 'warning');
      return;
    }
    BluetoothManager.updateStatus('명령 실행 중...', STATUS_COLORS.ORANGE);

    const fireAndForget = this._isFireAndForget(command);

    try {
      await BluetoothManager.sendData(command, !fireAndForget);
      if (DEBUG) Logger.add(`[완료] ${command}`, 'info');
    } catch (error) {
      // ... 에러 처리 ...
    }

    // 송신 간격 가드.
    // - fire-and-forget: UART(9600 baud)/BLE 버퍼 오버플로우 방지 최소 마진.
    // - 응답 대기: 이미 라운드트립을 거쳤으므로 가드 단축.
    const cooldown = fireAndForget ? 40 : 20;
    await new Promise(resolve => setTimeout(resolve, cooldown));
  }
}

```

효과

- 출력 명령(LED, SERVO 연속, DC 연속 등): BLE 라운드트립과 100 ms cooldown이 모두 사라짐 → 명령당 약 40 ms.
- 응답 대기 명령: cooldown만 100 ms→20 ms 단축. 의미적 동기화는 그대로 유지.

2.4 Pico 측 변경 --- Pico/main.py

웹의 분류와 동일 기준으로 Pico에서도 응답 송신 여부를 결정한다. 또한 단일 chunk 명령에 대한 무조건 50 ms 대기를 제거한다.

NO_RESPONSE_CMDS 및 _needs_response

```
class AresRover:
    # 응답 송신 생략 명령 (Web/commandexecutor.js의 FIRE_AND_FORGET_HEADS와 동기화).
    # 즉시 완료되는 출력 명령은 응답 라운드트립을 생략하여 처리량을 높이고,
    # 응답 매칭 어긋남(LED_ON 응답이 다음 명령 슬롯에 잘못 들어가는 현상)을 차단한다.
    NO_RESPONSE_CMDS = (
        "LED_ON", "LED_OFF", "MAIN_LED_ON", "MAIN_LED_OFF",
        "MSG", "CLEAR_DISPLAY",
        "SERVO_FORWARD", "SERVO_BACKWARD", "SERVO_LEFT", "SERVO_RIGHT", "SERVO_STOP",
        "DC_FORWARD", "DC_BACKWARD", "DC_STOP",
        "GUN_FIRE",
    )

    def _needs_response(self, data):
        """응답 송신 여부. fire-and-forget 명령은 False (NO_RESPONSE_CMDS 참조)."""
        if data.startswith("["):
            # LED 패턴 [v0 v1 v2 v3 v4]
            return False
        head = data.split(",", 1)[0]
        return head not in self.NO_RESPONSE_CMDS
```

_read_uart_line() 폴링 방식 교체

기존 구현은 매 호출마다 `utime.sleep_ms(RECEIVE_TIMEOUT_MS)`로 50 ms를 무조건 대기한 뒤 두 번째 `read`를 수행했다. `newline`이 이미 도착한 단일 chunk 명령에도 적용되어 처리량 천장이 약 16--17 cmd/s에 묶여 있었다. 폴링으로 교체하여 `newline` 발견 즉시 종료하도록 한다.

```
def _read_uart_line(self):
    """UART에서 완전한 라인 읽기.
    newline 발견 즉시 종료 - 단일 chunk 명령에서 무조건 50ms를 기다리지 않는다.
    멀티 chunk 명령(LED 패턴, SYS_SET 등)은 RECEIVE_TIMEOUT_MS까지 폴링하여 안전.
    """
    if not self.uart.any() and not self.rx_buffer:
        return None

    start = utime.ticks_ms()
    while utime.ticks_diff(utime.ticks_ms(), start) < RECEIVE_TIMEOUT_MS:
        if self.uart.any():
            chunk = self.uart.read()
            if chunk:
```

```

        self.rx_buffer += chunk.decode('utf-8', 'ignore')

    # 버퍼 오버플로우 방지
    if len(self.rx_buffer) > MAX_BUFFER_SIZE:
        print("[UART] 버퍼 오버플로우, 초기화")
        self.rx_buffer = ""
        return None

    # newline 즉시 종료
    if '\n' in self.rx_buffer:
        newline_idx = self.rx_buffer.index('\n')
        complete_line = self.rx_buffer[:newline_idx].strip()
        self.rx_buffer = self.rx_buffer[newline_idx + 1:]
        return complete_line
    else:
        utime.sleep_ms(2)

    # 타임아웃: 라인 종료자 없는 부분 데이터 반환
    if self.rx_buffer:
        data = self.rx_buffer.strip()
        if data and not data.endswith(','):
            self.rx_buffer = ""
            return data

    return None

```

run()의 응답 조건부 송신

```

# 변경 전
else:
    response = self._process_command(data)
    self._send_response(response)

# 변경 후
else:
    response = self._process_command(data)
    if self._needs_response(data):
        self._send_response(response)

```

명령 종류	변경 전	변경 후
LED_ON 등 fire-and-forget	약 250 ms	약 40 ms
BUZZER_ON, F, D 등 blocking	$D + 250$ ms	$D + 20$ ms
DISTANCE/MAGNET 등 값 반환	약 250 ms	RTT + 20 ms
Pico 처리량(단일 chunk)	$\approx 16\text{--}17$ cmd/s	$\approx 100^+$ cmd/s

표 3: 명령 부류별 종단 간 지연 비교 (단위: ms).

2.5 기대 효과 (정량)

추가로 응답 매칭 어긋남(이전 fire-and-forget 명령의 응답이 다음 응답 대기 명령의 pendingResolve 슬롯에 잘못 들어가는 코너 케이스) 위험이 구조적으로 제거된다. Pico에서 응답 자체를 보내지 않으므로 침범할 응답이 존재하지 않는다.

변경된 두 파일은 같이 갱신되어야 한다. 한쪽만 갱신되면:

- Web만 갱신: 출력 명령에 응답 대기를 안 하지만 Pico는 여전히 응답을 보냄 → BLE notify 누적, 응답 매칭 어긋남 위험 잔존.
- Pico만 갱신: Web이 출력 명령에 응답 대기를 하지만 Pico가 응답을 안 보냄 → 매 명령 5초 timeout.

3 2026-05-20 --- 재적용과 부팅 안정화

3.1 재적용 배경

전일 적용된 BLE 파이프라인 최적화(7e5b93f)는 적용 직후 BLE 연결 자체는 이루어지지만 GET_STATE 등 모든 명령이 응답하지 않는 현상이 보고되어 110bb27로 revert 되어 있었다.

재현 후 원인 분석한 결과, BLE 파이프라인 최적화 자체는 문제가 없었다. 진짜 원인은 Pico/main.py의 boot()에서 OLED 초기화 실패 시 robot.oled = None 상태인데 None.fill(0)이 호출 되어 AttributeError가 발생하고, 이 예외가 메인 루프 진입을 원천적으로 막아버린 데 있었다. 즉 펌웨어 부팅 단계의 무관한 크래시가 BLE 파이프라인 변경의 효과를 가린 것이다.

3.2 관찰의 단서 --- BLE 연결은 되는데 응답이 없다

BLE 모듈(BT05/HM-10)은 Pico의 UART에 연결된 외부 모듈로, BLE 페어링/연결은 모듈 자체 펌웨어가 처리하고 Pico는 UART로 데이터만 주고받는다. 따라서 Pico의 main.py가 boot에서 죽어도 BLE 링크는 멀쩡하다. 외부에서 보이는 증상은 “연결은 되는데 어떤 명령도 응답하지 않는다”로, 이는 정확히 Pico가 메인 루프에 진입조차 못한 상태의 결과이다.

3.3 수정 --- OLED None 가드

```
# 변경 전 (Pico/main.py boot())
self.robot.oled.fill(0)
self.robot.oled.text("ARES READY", 0, 0)
self.robot.oled.show()

# 변경 후
if self.robot.oled is not None:
    self.robot.oled.fill(0)
    self.robot.oled.text("ARES READY", 0, 0)
    self.robot.oled.show()
```

가드 추가 후 OLED 모듈이 비활성(또는 I2C 초기화 실패) 상태여도 boot()가 완료되며, 메인 루프가 정상 진입한다. 이 가드는 다른 하드웨어 모듈에 적용된 if robot.module: 패턴과 일관된 방어 코드이다.

가드 적용 후 5월 19일의 BLE 최적화를 동일 형태로 재적용하였고 (600243a), 정상 동작을 확인하였다.

4 2026-05-20 --- SYS_SET 핸들러 누락 수정

4.1 증상

대시보드에서 “최대 속도” 슬라이더를 낮추고 “Pico에 업로드” 버튼을 누르면 “설정이 Pico에 업로드되었습니다!” 라는 알림이 떴지만, 다른 탭으로 갔다가 돌아오면 슬라이더가 다시 80%로 복귀했다.

4.2 원인

Pico/process_data.py의 명령 디스패처에서:

```
elif data.startswith("SYS_SET"):
    return self._handle_sys_set(data)
```

호출 대상인 _handle_sys_set 메서드가 정의되어 있지 않았다. 메서드 헤더(def _handle_sys_set(self, data:))가 빠져 있어, 핸들러 본문이 직전의 _handle_set_module의 return 뒤에 떨어져 도달 불가능한 코드(dead code)가 되어 있었다. SYS_SET 명령이 도달하면 AttributeError가 발생 하여 main.py의 예외 처리에 잡혀 "ERROR"가 반환되고, system.json은 갱신되지 않았다. Web 측은 응답을 확인하지 않고 즉시 alert를 띄우므로 사용자에게는 "저장된 것처럼" 보였다.

4.3 수정

```
def _handle_sys_set(self, data):
    """시스템 설정 저장: SYS_SET,max_speed,col_dist,auto_stop,name"""
    try:
        parts = data.split(',')
        if len(parts) >= 5:
            max_speed = parts[1]
            col_dist = parts[2]
            auto_stop = parts[3]
            name = ",".join(parts[4:])
            sys_config.save_config(max_speed, col_dist, auto_stop, name)
            return 1
    except Exception as e:
        print(f"SYS_SET 오류: {e}")
    return 0
```

수정 후 슬라이더 값이 system.json에 영구 저장되고, Pico 재부팅 후에도 유지된다.

Web 측 saveSystemSettings()는 SYS_SET 응답을 검증하지 않고 무조건 성공 alert를 띄운다. 후속 개선 항목으로 sendData(cmd, true)로 응답 대기 후 결과를 분기하는 작업을 분리하여 진행한다.

5 2026-05-20 --- DC 모터 H-브릿지 비대칭 진단과 듀얼 PWM 대칭화

5.1 증상의 변천

1. 초기: 전진이 후진보다 명백히 약함(특히 max_speed가 높을수록 격차가 큼).
2. 단순 PWM 반전 적용 후: 80 %에서는 양방향 정상이나, 슬라이더 55 %이하에서 후진이 시동되지 않음.
3. 후진에 시동 데드존 lift 적용(56 %): 시동은 되지만 후진 토크가 56--60 % 구간에서 약함.
4. 모터 리드를 물리적으로 뒤집는 진단 실험: 회전 방향만 반대로 바뀌고 전·후진 속도 특성은 함수 호출 기준 그대로 유지됨.

5.2 진단 결과 --- 비대칭은 모터가 아니라 드라이버 입력 토폴로지

기존 코드는 DIR + PWM 한 쌍으로 H-브릿지를 구동하였다. 이때 드라이버 칩의 IN1/IN2 두 입력 중 한쪽은 디지털 DIR로 고정되고 다른 쪽만 PWM이 들어가므로, 두 회전 방향이 서로 다른 PWM 변조 모드로 동작한다.

모드	동작	특성
DIR=1, IN2 PWM (전진)	drive↔brake 변조	권선 전류 연속, 시동 우수, 듀티가 작을수록 토크 큼
DIR=0, IN2 PWM (후진)	drive↔coast 변조	권선 전류 불연속, 시동 데드존 존재, 약 56 % 이하에서 시동 불가

표 4: 단일 DIR+PWM 토폴로지에서 회전 방향별 PWM 변조 모드.

모터 리드를 뒤집어도 함수 단위의 비대칭이 그대로 유지된 실험 결과가 이 가설을 결정적으로 확증한다. 비대칭은 드라이버 입력 측 패턴에 귀속되며, 모터 권선이나 기어박스의 마찰 비대칭은 아니다.

5.3 듀얼 PWM brake-modulated 구동

RP2040에서 GP8/GP9는 같은 PWM 슬라이스(PWM4)의 A/B 채널이다. 두 핀을 모두 PWM으로 설정한 뒤, 한쪽을 PWM_MAX로 고정하고 반대쪽을 PWM_MAX - speed로 변조하면 양방향 모두

brake-modulated가 되어 토크 곡선이 대칭화된다.

```
# Pico/dcmotor.py
from machine import Pin, PWM
from pins import DCMOTOR_DIR_PIN, DCMOTOR_PWM_PIN

IN1_PIN = DCMOTOR_DIR_PIN # GP8 (PWM4A) - 이름은 레저시 호환 유지
IN2_PIN = DCMOTOR_PWM_PIN # GP9 (PWM4B)

PWM_FREQUENCY = 20000
PWM_STOP = 0
PWM_MAX = 65535

class KSDCMotor:
    """DC 모터 컨트롤러 (양방향 brake-modulated H-브릿지)."""

    def __init__(self):
        self.IN1 = PWM(Pin(IN1_PIN))
        self.IN1.freq(PWM_FREQUENCY)
        self.IN1.duty_u16(PWM_STOP)

        self.IN2 = PWM(Pin(IN2_PIN))
        self.IN2.freq(PWM_FREQUENCY)
        self.IN2.duty_u16(PWM_STOP)

        self._is_running = False

    def dc_forward(self, speed):
        """전진: IN1 고정 HIGH, IN2를 (PWM_MAX-speed)로 변조."""
        if speed <= 0:
            self.dc_stop(); return
        self.IN1.duty_u16(PWM_MAX)
        self.IN2.duty_u16(PWM_MAX - speed)
        self._is_running = True

    def dc_backward(self, speed):
        """후진: IN2 고정 HIGH, IN1을 (PWM_MAX-speed)로 변조."""
        if speed <= 0:
            self.dc_stop(); return
        self.IN1.duty_u16(PWM_MAX - speed)
        self.IN2.duty_u16(PWM_MAX)
        self._is_running = True
```

```
def dc_stop(self):
    """정지 (coast: 둘 다 LOW)."""
    self.IN1.duty_u16(PWM_STOP)
    self.IN2.duty_u16(PWM_STOP)
    self._is_running = False
```

5.4 기대 효과

슬라이더	전진 실효 듀티	후진 실효 듀티	비고
100 %	100 %	100 %	풀파워, 양방향 동일
80 %	80 %	80 %	동일
50 %	50 %	50 %	동일
20 %	20 %	20 %	동일 (시동은 모터 마찰 한계에 의존)
0 %	0	0	coast

표 5: 듀얼 PWM brake-modulated 구동의 양방향 대칭 출력.

시동 데드존(BACKWARD_DEADZONE) lift 같은 보정 hack이 더는 필요하지 않다. 소프트웨어로 우회하던 비대칭을 펌웨어 구성 변경으로 근원에서 제거한 사례이다. 회로 배선이나 드라이버 교체 없이 펌웨어만으로 해결되었다는 점이 특히 가치 있다.

5.5 잔여 사항

- pins.py 및 system_config.py의 핀 이름 (DCMOTOR_DIR_PIN, pin_dcmotor_dir)은 의미상 "DIR"이 아니지만 외부 API(SET_PIN 명령 등)와의 호환을 위해 레거시 명칭을 유지했다. 향후 정리 시 IN1/IN2 또는 A/B로 명명 통일이 권장된다.
- 모터의 정적 마찰 한계로 약 20 % 이하의 매우 낮은 듀티에서는 여전히 회전하지 않을 수 있다. 이는 회로/펌웨어 문제가 아닌 모터 물성으로, 학습 환경에서는 max_speed를 30 % 이상에서 운영할 것을 권장한다.

6 2026-05-20 --- Web Bluetooth와 HTTPS 서버 사용 사유

6.1 관찰

Web/ks01.html은 file:// 프로토콜로 직접 열어도 BLE 연결이 가능했다. 반면 정식 학습용 Web/main.html은 file://에서 연결되지 않고, http://localhost 또는 https:// 호스팅 환경에서만 동작한다.

6.2 원인 --- Web Bluetooth가 아니라 ES module의 same-origin 정책

직관적으로는 “Web Bluetooth가 file://에서 secure context로 인정되지 않기 때문”으로 생각하기 쉽지만, 실제 원인은 다르다. 두 파일의 스크립트 로딩 방식이 결정적으로 다르다.

파일	스크립트 로딩 방식	file://에서 동작?
ks01.html	모든 코드가 인라인 <code><script>...</script></code>	정상 동작
main.html	<code><script type="module" src="main.js"></code> , main.js는 bluetooth.js 등 다수 모듈을 import	차단 (모듈 fetch 실패)

표 6: 스크립트 로딩 방식과 file:// 동작 거부.

Chromium 기반 브라우저는 file://을 secure context로 인정하므로 navigator.bluetooth API 자체는 file://에서도 호출 가능하다. 그러나 `<script type="module">`은 같은 출처 (same-origin) 정책을 따르는데, 모든 file:// URL은 같은 디렉터리에 있더라도 origin: null로 취급되어 서로 다른 출처로 간주된다. 따라서 main.js를 비롯한 ES 모듈 fetch가 CORS 정책으로 차단되고, 콘솔에는 다음과 같은 오류가 기록된다.

```
Access to script at 'file:///.../main.js' from origin 'null'
has been blocked by CORS policy: Cross origin requests are
only supported for protocol schemes: http, data, isolated-app,
chrome-extension, chrome-untrusted, https, chrome.
```

ES 모듈이 로드되지 않으면 BluetoothManager 코드는 실행될 기회조차 없다. 즉 사용자가 본 “BLE가 연결되지 않는” 증상은 Web Bluetooth API의 보안 제한이 아니라 BLE 코드가 아예 실행되지 않은 결과이다. 증상이 같아 보였을 뿐이다.

ks01.html이 file://에서 동작하는 것은 데스크톱 Chrome 환경에 한정될 가능성이 크다. Android Chrome은 file://에서 Web Bluetooth를 더 엄격히 제한하는 경우가 있으며, 모바일 배포를 가정한다면 이 경로에 의존해서는 안 된다.

6.3 해결 옵션과 권장

방법	설명	권장
A. <code>python -m http.server</code>	localhost는 secure context로 인정되고 ES 모듈도 같은 출처에서 fetch 가능	개발 단계
B. HTTPS 호스팅 (GitHub Pages 등)	어디서나 동작, 모바일 포함. file:// 의존 사라짐	학생 배포
C. 단일 HTML로 번들링(esbuild/rollup)	main.js와 모듈을 인라인 IIFE로 합쳐 ks01.html과 같은 형태	오프라인 단일 파일이 꼭 필요한 경우
D. ES module 제거, 전역 객체로 통합	import/export 제거 후 인라인 <script>로 합치기	권장하지 않음 (모듈성 손실 큼)

표 7: main.html의 file:// 미동작에 대한 해결 옵션.

학생 배포를 전제하면 B(HTTPS 호스팅)가 가장 깔끔하다. 모바일 브라우저 호환성, 즐겨찾기/PWA 설치, 자동 업데이트 측면 모두 유리하다. ks01.html처럼 단일 파일 배포가 정책적으로 요구되는 경우에는 C(번들링)가 정공법이다. 어느 쪽이든 file:// 직접 열기에 의존하는 워크플로우는 장기적으로 권장되지 않는다.

7 2026-05-20 (오후) --- BATCH “한꺼번에 실행” 명령 블록과 파이프라인 정착

BLE 파이프라인을 출력 명령에 대해 fire-and-forget으로 최적화한 결과 단일 명령당 약 40 ms의 Web 측 cooldown까지는 줄였지만, 명령이 수십 개에 달하는 시퀀스(카운트다운, 사이렌, LED 시퀀스)에서는 누적 cooldown이 여전히 수 초에 이르는 문제가 남았다. 본 절은 명령을 묶어 한 번의 BLE 라운드트립으로 보내는 BATCH 메커니즘과, 이를 클론 BT05/HM-10 환경에서 안정적으로 동작시키기 위해 진행한 단계별 수정을 정리한다.

7.1 동기와 설계

명령을 묶을 컨테이너 블록을 Blockly에 도입하고, 안에 들어간 자식 블록들의 명령 문자열을 |로 이어 BATCH;cmd1|cmd2|cmd3 형태의 단일 명령으로 Pico에 보낸다. Pico는 자체 큐로 순차 실행하고 마지막에 단일 응답을 돌려준다.

이득	의존 매개변수	BATCH로 달성
명령당 cooldown 누적 제거	Web cooldown 정책	단일 cooldown(20 ms)으로 압축
SLEEP/BUZZER_ON 타이밍 정확도	Pico 내부 처리	네트워크 지연 영향 제거
응답 매칭 어긋남 위험 제거	단일 ACK 구조	sub-command별 ACK 없음
BLE 송신 시간 단축	체크 수	부수적 (CHUNK_DELAY에 영향)

표 8: BATCH 도입의 본질적 이득 분해.

7.2 Blockly 블록 디자인과 차단 정책

새 블록 batch_block은 단일 statement input을 가지는 컨테이너이며 보라색(#8E44AD)으로 시각적으로 두드러진다. 안에 들어가면 안 되는 블록(센서 값 반환, 반복/조건/변수, 함수 호출, 자기 자신)은 미리 집합으로 명시한다.

```
// Web/blocklyconfig.js
export const BATCH_FORBIDDEN_TYPES = new Set([
  // 값 반환
  'check_distance', 'check_magnetic', 'pico_check_device',
  // 제어 흐름
  'controls_if', 'controls_whileUntil', 'controls_repeat_ext',
  // 변수/함수 (제어 흐름은 Web 측 책임)
  'variables_set', 'assign_variable', 'math_change',
  'procedures_callnoreturn', 'procedures_callreturn',
  'procedures_defnoreturn', 'procedures_defreturn',
```

```
// 중첩 금지
'batch_block',
]);
```

학생이 금지 블록을 batch 안에 떨어뜨리면 setOnChange 콜백이 자식을 순회해 노란 □ 경고를 즉시 띄운다. 실행 시점에도 commandexecutor.js의 _processBatch가 한 번 더 검증하여 금지 블록 발견 시 빨간 로그와 함께 실행을 중단한다(이중 방어).

7.3 프로토콜과 핵심 구현

```
// Web/commandexecutor.js
async _processBatch(block) {
  const commands = [];
  let cur = block.getInputTargetBlock('D0');
  while (cur) {
    if (BATCH_FORBIDDEN_TYPES.has(cur.type)) {
      Logger.add(`[오류] '${cur.type}' 블록은 [한꺼번에 실행] 안에 넣을 수 없습니다.`, 'error');
      state.isExecuting = false;
      return;
    }
    const cmd = this.generateCommand(cur);
    if (cmd) commands.push(cmd);
    cur = cur.getNextBlock();
  }
  if (commands.length === 0) return;
  const payload = `BATCH;${commands.join('|')}`;
  await BluetoothManager.sendData(payload, true); // 응답 대기 1회
  await new Promise(r => setTimeout(r, 20));
}
```

```
# Pico/process_data.py
def _handle_batch(self, data):
    """BATCH;cmd1|cmd2|cmd3 - 기존 디스패처로 차례 실행 후 단일 응답."""
    try:
        body = data[len("BATCH;"):].split('|')
        for cmd in body:
            cmd = cmd.strip()
            if not cmd:
                continue
            self.process(cmd)
```



```

    return 1
except Exception as e:
    print(f"BATCH 오류: {e}")
    return 0

```

7.4 진행 중 발견된 안정화 이슈와 단계별 수정

초기 구현은 단일 청크에 들어가는 매우 짧은 BATCH에서만 정상 동작했고, 일반 학습 시나리오(LED 2개 + SLEEP 등)는 멀티 청크로 전송되며 다음 5단계의 문제를 차례로 드러냈다.

1단계 --- 부분 라인 반환으로 인한 BATCH 절단

`_read_uart_line()`이 `RECEIVE_TIMEOUT_MS` 동안 `newline`을 못 만나면 실패로 끝나지 않은 부분 데이터를 반환하는 옛 동작 때문에, 첫 청크 "BATCH;LED_ON,0,1;LED"가 그대로 디스패처에 넘어가 LED_ON,0,1만 실행되고 나머지가 잘려나갔다. 부분 반환을 제거하고 `buffer`를 유지한 채 `None`을 반환하도록 변경했다.

```

# Pico/main.py
def _read_uart_line(self):
    if not self.uart.any() and not self.rx_buffer:
        return None
    start = utime.ticks_ms()
    while utime.ticks_diff(utime.ticks_ms(), start) < RECEIVE_TIMEOUT_MS:
        if self.uart.any():
            chunk = self.uart.read()
            if chunk:
                self.rx_buffer += chunk.decode('utf-8', 'ignore')
            if len(self.rx_buffer) > MAX_BUFFER_SIZE:
                print("[UART] 버퍼 오버플로우, 초기화")
                self.rx_buffer = ""
                return None
            if '\n' in self.rx_buffer:
                idx = self.rx_buffer.index('\n')
                line = self.rx_buffer[:idx].strip()
                self.rx_buffer = self.rx_buffer[idx + 1:]
                return line
        else:
            utime.sleep_ms(2)
    # 타임아웃 시 buffer 유지, 다음 호출에서 이어받음

```

```
return None
```

2단계 --- 청크 손실 진단과 페이싱 조정

원인 추적 끝에 BT05/HM-10 클론에서 청크 사이 간격이 BLE connection interval(30--70 ms)과 겹쳐 청크가 일부 손실됨을 확인했다. AT 명령으로 connection interval을 단축하는 정식 경로는 클론에서 미지원이므로 Web 측 페이싱을 보수적으로 조정했다.

매개변수	변경 전	정착값
Web/constants.js : CHUNK_DELAY	50 ms	100 ms
Pico/main.py : RECEIVE_TIMEOUT_MS	50 ms	500 ms

표 9: 클론 BLE 모듈을 위한 페이싱 정착값.

3단계 --- GATT ACK 왕복 제거

writeValueWithResponse는 매 청크마다 GATT 수준 ACK를 기다리므로 멀티 청크 송신 시간이 누적된다. characteristic이 지원하면 writeValueWithoutResponse를 우선 사용하고, 그렇지 않으면 기존 방식으로 fallback하도록 했다.

```
// Web/bluetooth.js
const useWithoutResponse =
  state.characteristic.properties &&
  state.characteristic.properties.writeWithoutResponse;
// for-loop 내부에서:
if (useWithoutResponse) {
  await state.characteristic.writeValueWithoutResponse(chunk);
} else {
  await state.characteristic.writeValueWithResponse(chunk);
}
```

4단계 --- 응답 race condition 수정

BATCH가 Pico에서 매우 빠르게 처리되어 마지막 청크 송신 완료 직후 응답이 도착하는데, pendingResolve가 송신 후에 설치되어 있어 응답이 state.pendingResolve === null 상태에서

버려졌다. 5초 RESPONSE_TIMEOUT 후 명령이 실패한 것처럼 보였지만 Pico에서는 이미 두 LED 모두 점등되어 있었다. promise 설치를 송신 시작 전으로 옮겨 race를 달았다.

```
// Web/bluetooth.js - sendData 발췌
let responsePromise = null;
if (waitForResponse) {
  responsePromise = new Promise((resolve, reject) => {
    state.pendingResolve = resolve;    // ← 송신 전 설치
    state.pendingReject = reject;
    state.pendingTimeout = setTimeout(() => {
      state.pendingResolve = null;
      state.pendingReject = null;
      state.pendingTimeout = null;
      reject(new Error('응답 시간 초과'));
    }, BLUETOOTH_CONFIG.RESPONSE_TIMEOUT);
  });
}
// ... 청크 송신 ...
return waitForResponse ? responsePromise : 'OK';
```

5단계 --- 단일 청크 명령의 마지막 CHUNK_DELAY 제거

CHUNK_DELAY=100ms로 정착한 뒤 일반 LED 명령처럼 단일 청크 명령조차 매번 100ms를 추가로 기다리는 부작용이 관찰되었다. 송신 루프가 마지막 청크 송신 후에도 무조건 CHUNK_DELAY를 실행했기 때문이다. 청크 사이에만 페이싱이 적용되도록 guard를 추가했다.

```
for (let i = 0; i < encodedData.length; i += BLUETOOTH_CONFIG.MAX_CHUNK_SIZE) {
  const chunk = encodedData.slice(i, ...);
  if (useWithoutResponse) await state.characteristic.writeValueWithoutResponse(chunk);
  else await state.characteristic.writeValueWithResponse(chunk);
  // 청크 사이에만 페이싱 - 마지막 청크 후엔 즉시 종료
  if (i + BLUETOOTH_CONFIG.MAX_CHUNK_SIZE < encodedData.length) {
    await this.delay(BLUETOOTH_CONFIG.CHUNK_DELAY);
  }
}
```

명령 형태	변경 전	BATCH 도입 직후	BATCH 정착 후
단일 청크 출력 (LED_ON 등)	약 40 ms	약 140 ms (마지막 CHUNK_DELAY 100 ms 가산)	약 40 ms (마지막 CHUNK_DELAY guard로 회복)
멀티 청크 시퀀스 100명령 누적 cooldown	약 4초	---	단일 cooldown으로 압축
SLEEP/BUZZER_ON 타이밍 정확도	네트워크 지연 영향	---	Pico 내부 처리, 정확
응답 매칭 어긋남	가능성 있음	가능성 있음	단일 응답이라 구조적 제거

표 10: BATCH 도입 전후의 명령별 중단 효과.

7.5 최종 효과

이번 작업은 두 가지 가치가 함께 간다. 첫째, 카운트다운/사이렌 같은 시간 의존 시퀀스가 시각적으로 부드러워졌다. 둘째, 학생이 "하나씩 결과를 확인하고 싶은 동작"과 "빠르게 한 번에 보내고 싶은 동작"을 Blockly 블록 위에 명시적으로 표현하게 되었다. 추상적으로는 원격 호출(RPC)과 원격 스크립트 송신의 차이를 초등 수준에서 체험하는 학습 도구이기도 하다.

7.6 BATCH 관련 잔여 사항

- RESPONSE_TIMEOUT=5 초는 BATCH 안의 누적 SLEEP이 5 초를 넘으면 timeout이 먼저 도착한다. 후속 개선으로 BATCH의 명령 합에 비례한 동적 timeout 산정 또는 BATCH 전용 더 긴 timeout 도입을 검토한다.
- BATCH 안 명령을 단축 토큰(예: LED_ON,0,1 → L+0) 으로 packing하면 한 청크에 더 많은 명령이 들어가 송신 시간을 더 단축할 수 있다. 디스패처 호환성 작업이 필요하므로 별도 분리.

8 잔여 위험 및 후속 작업

8.1 Web 측 saveSystemSettings() 응답 검증

SYS_SET 송신 후 무조건 성공 alert를 띄우는 현행 동작은 펌웨어에서 실제로 저장 실패한 경우를 사용자에게 가린다. 응답 대기 명령으로 승격하여 "1"/"0" 또는 "OK"/"ERROR" 구분 후 분기 alert를 띄우는 작업이 권장된다.

8.2 UART baud rate

가장 본질적인 처리량 천장은 BLE 모듈 ↔ Pico 사이의 9600 baud UART이다. HM-10/BT05의 AT 명령(AT+BAUD8 등)으로 38400 또는 115200 baud로 상향하고 Pico/main.py의 UART_BAUDRATE를 동기화하면 byte당 전송 시간을 4--12배 단축할 수 있다. BLE 모듈 펌웨어 설정과 Pico 코드 동시 수정이 필요하므로 별도 작업으로 분리한다.

8.3 호스팅 환경 정비

Web/main.html의 ES module 의존을 그대로 두면서 학생 배포를 원활하게 하려면 GitHub Pages 등 HTTPS 호스팅을 정식 채택할 필요가 있다. 도메인/배포 자동화/PWA manifest 추가가 후속 작업 후보이다.

8.4 권장 회귀 테스트

1. 카운트다운 --- LED_OFF 5개 연속 소등. 명령 사이 체감 지연이 거의 사라져야 한다.
2. LED 패턴 + 거리 센서 혼합 --- [v0 v1 v2 v3 v4] 직후 DISTANCE. 거리값이 1 같은 LED 응답으로 오염되지 않는지 확인.
3. 부저 사이렌 --- BUZZER_ON,400,0.5 와 BUZZER_ON,800,0.5의 반복. 타이밍 정확성이 유지되어야 한다.
4. 시간 제한 모터 --- DC_tFORWARD,N, DC_tBACKWARD,N의 N초 동기화가 유지되는지 확인.
5. DC 모터 양방향 대칭 --- max_speed를 30, 50, 80, 100 %로 바꿔가며 전진/후진 속도가 거의 같게 느껴지는지 확인.
6. 대시보드 max_speed 영구 저장 --- 슬라이더 조정 → 업로드 → 다른 탭 이동 → 대시보드 복귀 시 값이 유지되는지 확인. Pico 재부팅 후에도 유지되어야 한다.

7. BATCH 카운트다운 --- “한꺼번에 실행” 블록 안에 LED 5 → 1 점등/소등 + SLEEP,1을 넣고 실행. 한 번의 [전송] BATCH;... 로그와 단일 [묶음 완료] 응답이 떴어야 하며, LED 시퀀스가 끊김 없이 매끄러워야 한다.
8. BATCH 금지 블록 차단 --- 거리 센서/반복/조건 블록을 “한꺼번에 실행” 안에 끌어놓을 때 노란 □ 경고가 즉시 표시되고, 실행 시 빨간 오류 로그와 함께 중단되는지 확인.
9. 단일 청크 명령 속도 회복 --- 일반 LED_ON 단일 블록을 5~10회 연속 실행. 명령 사이 체감 지연이 CHUNK_DELAY 도입 전 수준(약 40 ms)으로 돌아왔는지 확인.

9 변경 파일 요약

일자	파일	변경 요지
05-19	Web/commandexecutor.js	FIRE_AND_FORGET_HEADS 집합 도입, sendCommand()에서 분기 처리, cooldown 분리(40 / 20 ms).
05-19	Pico/main.py	NO_RESPONSE_CMDS 클래스 변수, _read_uart_line() 폴링 방식으로 교체, _needs_response() 도입, run()에서 조건부 _send_response().
05-20	Pico/main.py	boot()의 OLED 호출 None 가드 추가. 05-19 BLE 최적화 재적용.
05-20	Pico/process_data.py	누락되어 있던 _handle_sys_set() 메서드 헤더 복구.
05-20	Pico/dcmotor.py	DIR+PWM 단일 변조에서 양 핀 모두 PWM으로 전환. dc_forward/dc_backward가 한쪽 핀을 PWM_MAX 고정, 다른 쪽을 PWM_MAX-speed로 변조하여 양방향 brake-modulated 대칭 출력.
05-20	Web/blocklyconfig.js	batch_block JSON 정의 + BATCH_FORBIDDEN_TYPES + attachBatchBlockValidator().
05-20	Web/main.js	attachBatchBlockValidator(Blockly) 호출 등록.
05-20	Web/commandexecutor.js	processBlock에서 batch_block 분기, _processBatch 헬퍼 (자식 평탄화 + 금지 블록 차단 + 단일 BATCH;... 송신).
05-20	Web/main.html	toolbox에 "한꺼번에" 카테고리 추가.
05-20	Web/constants.js	CHUNK_DELAY 50 → 100 ms.
05-20	Web/bluetooth.js	writeValueWithoutResponse 우선 사용 (fallback 포함), pendingResolve를 송신 전 설치 (race 해소), 마지막 청크 후 CHUNK_DELAY guard.
05-20	Pico/process_data.py	BATCH; 디스패처 + _handle_batch() (파이프 분리 후 기존 process() 재진입).
05-20	Pico/main.py	RECEIVE_TIMEOUT_MS 50 → 500 ms, 타임아웃 시 부분 데이터 반환 제거(buffer 유지).

표 11: 2026-05-19 - 05-20 변경 범위.

일자	커밋	메시지
05-19	7e5b93f	Optimize BLE pipeline: fire-and-forget for output commands
05-19	110bb27	Revert ``Optimize BLE pipeline: fire-and-forget for output commands''
05-20	600243a	Re-apply BLE pipeline optimization with OLED boot crash fix
05-20	de9cffc	Fix DC motor: forward PWM asymmetry and missing SYS_SET handler
05-20	5780485	Drive H-bridge with dual PWM for symmetric DC motor output
05-20	4fccfe7	Add ``한꺼번에 실행'' (BATCH) block to bundle commands into one BLE write
05-20	c6d57ae	Hold UART buffer across timeouts so multi-chunk BATCH stays intact
05-20	8d341af	Settle BATCH path: drop debug prints, switch BLE writes to without-response
05-20	21c4e26	Fix BLE response race: install pendingResolve before sending
05-20	233f185	Skip CHUNK_DELAY after the final chunk so single-chunk commands stay fast

표 12: 관련 커밋 (main 브랜치).